

Semana 6 — Clase Práctica en Laboratorio

Algoritmos de Ordenamiento, Archivos de Texto y Procesamiento de Datos Estructurados

Introducción

Los problemas de esta semana integran los **algoritmos de ordenamiento** (selección, inserción, burbuja, merge sort, quick sort) con el **manejo de archivos de texto** en Python. El eje temático es la implementación correcta de los algoritmos, su **comparación experimental** mediante medición de tiempos, y la aplicación a problemas realistas de **procesamiento de datos persistentes**: análisis de texto y datos estructurados almacenados en archivos.

Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa modular** de la Semana 4: cada algoritmo se implementa como una **función** con sección declarativa (docstring), precondiciones y postcondiciones.

Los siete problemas están organizados en cuatro bloques con **conexiones deliberadas**: los Problemas 1 y 2 implementan y comparan experimentalmente los algoritmos de ordenamiento cuadráticos y merge sort; el Problema 3 agrega quick sort y diseña un experimento de medición de tiempos completo; el Problema 4 aplica ordenamiento a un problema clásico de lingüística computacional (frecuencias de palabras y Ley de Zipf); el Problema 5 genera y procesa datos comerciales estructurados en archivos de texto, integrando generación pseudoaleatoria, búsqueda binaria, ordenamiento y E/S de archivos. Los Problemas 6 y 7 son desafíos de extracursado que profundizan el análisis.

Recordatorio: Modelo de programa modular

*Cada función de ordenamiento y de procesamiento de archivos incluye su sección declarativa como **docstring** (descripción, precondición, postcondición), accesible mediante `help()`. La sección algorítmica se organiza en Prólogo / Resolución / Epílogo.*

Recordatorio: `for/range()` vs. `while`

*Norma de la Semana 3: **for** con **range()** para recorridos determinados (se sabe cuántas iteraciones); **while** para recorridos indeterminados (la condición de parada depende de los datos o de múltiples criterios simultáneos). Por ejemplo: el bucle externo de selección es `for i in range(n-1)`; el bucle interno de inserción es `while j >= 0 and datos[j] > clave`.*

Módulos `random` y `time` de Python

Para generación pseudoaleatoria reproducible: `random.seed(42)` al inicio. Funciones útiles: `random.randint(a, b)`, `random.sample(population, k)`, `random.choices(population, weights, k)`, `random.shuffle(lista)`. Para medición de tiempos: `time.perf_counter()` (resolución en nanosegundos).

Bloque A: Implementación y validación de algoritmos de ordenamiento

Problema 1: Ordenamiento por selección, inserción y burbuja

Enunciado

Implementar tres funciones de ordenamiento in-place en Python: `ordenamiento_seleccion(datos)`, `ordenamiento_insercion(datos)` y `ordenamiento_burbuja(datos)`. Cada función recibe una lista mutable de enteros y la ordena in-place en forma no decreciente. El ordenamiento por burbuja debe incluir la optimización de parada temprana (bandera de intercambio).

A continuación, escribir un programa que:

- Genere una lista de $n = 20$ enteros pseudoaleatorios en el rango $[1, 100]$ con `random.seed(42)`. Mostrar la lista original.
- Para cada algoritmo, crear una copia de la lista original (`datos.copy()`), ordenarla e imprimir el resultado. Verificar que las tres funciones producen la misma lista ordenada.
- Agregar un contador de comparaciones y un contador de intercambios (o desplazamientos, en el caso de inserción) a cada función. Informar ambos contadores para cada algoritmo y comparar con las predicciones teóricas.

Resultado esperado (orientativo)

Algoritmo	Comparaciones	Intercambios/Desplaz.	Predicción teórica
Selección	190	≤ 19	$n(n-1)/2 = 190$ comp.
Inserción	~80–120	~80–120	$n(n-1)/4 \approx 95$ (promedio)
Burbuja	~150–190	~80–120	$n(n-1)/2$ sin optim.

Orientaciones para la resolución

Análisis: Los contadores permiten verificar empíricamente las fórmulas de complejidad. Selección siempre hace exactamente $n(n-1)/2$ comparaciones. Inserción y burbuja varían según el grado de desorden de los datos.

Diseño: Usar `for i in range(n - 1)` para los bucles externos determinados de selección e inserción, y `while` para el bucle externo de burbuja (condición compuesta con bandera) y el bucle interno de inserción (condición compuesta con datos). Consultar las implementaciones de la sección 2–4 del documento teórico.

Evaluación: Verificar que las tres funciones producen la misma salida. Verificar que selección da exactamente 190 comparaciones para $n = 20$.

Problema 2: Merge sort y comparación con algoritmos cuadráticos

Enunciado

Implementar las funciones `fusionar(izq, der)` y `merge_sort(datos)` siguiendo el diseño de la clase teórica. La función `merge_sort` retorna una **nueva** lista ordenada (no modifica la original).

A continuación, escribir un programa que:

- (a) Genere listas pseudoaleatorias de tamaños $n \in \{100, 500, 1000, 5000\}$ con `random.seed(42)` para cada tamaño.
- (b) Para cada n , mida el tiempo de ejecución de selección, inserción, burbuja y merge sort usando `time.perf_counter()`. Repetir 3 veces y promediar.
- (c) Presente una tabla de resultados y calcule el ratio Selección/MergeSort para cada n .
- (d) Analice: ¿El ratio crece cuando n se duplica? ¿Qué relación tiene con $O(n^2)/O(n \log n) = O(n/\log n)$?

Orientaciones para la resolución

Análisis: El objetivo es verificar empíricamente que merge sort escala mejor que los cuadráticos. El ratio $O(n^2)/O(n \log n) = O(n/\log n)$ crece con n : para $n = 100$, $n/\log_2(n) \approx 15$; para $n = 5000$, ≈ 408 .

Diseño: Estructura modular recomendada: una función `medir_ordenamiento(funcion, datos, repeticiones)` que retorna el tiempo promedio. Para los algoritmos in-place, pasar `datos.copy()` en cada repetición. Para merge sort, como retorna una nueva lista, pasar la original directamente.

Conexión con la jerarquía de memoria

Para $n = 5000$, inserción puede ser más rápida que merge sort a pesar de ser $O(n^2)$ teóricamente, porque su excelente localidad de caché compensa las comparaciones adicionales. Este fenómeno ilustra la diferencia entre complejidad teórica y rendimiento real — un tema que reaparece en la Semana 10.

Bloque B: Quick sort y experimento comparativo completo

Problema 3: Quick sort y laboratorio de medición de tiempos

Enunciado

Implementar la función `quick_sort(datos)` en su versión simplificada (no in-place), siguiendo la implementación de la sección 7.3 del documento teórico.

A continuación, diseñar y ejecutar un experimento completo que compare los cinco algoritmos (selección, inserción, burbuja, merge sort, quick sort) en tres escenarios:

Escenario A — Datos aleatorios: lista de n enteros generados con `random.sample(range(1, 10 * n), n)`.

Escenario B — Datos ya ordenados: la misma lista del Escenario A, pero ordenada previamente con `sorted()`.

Escenario C — Datos en orden inverso: la lista del Escenario A ordenada con `sorted(reverse=True)`.

Para cada escenario y $n \in \{500, 1000, 2000, 5000\}$, medir tiempos y presentar una tabla. Analizar especialmente:

- ¿Qué le pasa a inserción en el Escenario B (mejor caso)?
- ¿Qué le pasa a quick sort en el Escenario B con pivote = último (peor caso)? ¿Y si se elige pivote aleatorio?
- ¿Cuál algoritmo gana en cada escenario?

Orientaciones para la resolución

Análisis: El Escenario B revela las diferencias adaptativas: inserción es $O(n)$, burbuja con optimización es $O(n)$, selección sigue siendo $O(n^2)$, y quick sort con pivote último degrada a $O(n^2)$. Merge sort es $O(n \log n)$ en todos los casos.

Diseño: Para quick sort con pivote aleatorio, modificar la función reemplazando `pivote = datos[n - 1]` por `idx = random.randint(0, n - 1)` y luego intercambiar `datos[idx]` con `datos[n - 1]` antes de particionar. Incluir ambas versiones en la tabla del Escenario B.

Precaución con la profundidad de recursión

Para $n = 5000$ con datos ya ordenados y pivote último, quick sort genera 5000 niveles de recursión, superando el límite por defecto de Python (1000). Usar `import sys; sys.setrecursionlimit(10000)` al inicio del programa, o bien capturar el `RecursionError` e informar que el algoritmo no pudo completarse con esa configuración.

Bloque C: Archivos de texto y procesamiento de datos

Problema 4: Análisis de frecuencia de palabras y Ley de Zipf

Enunciado

Escribir un programa que lea un archivo de texto (por ejemplo, un capítulo de una obra literaria de dominio público descargado de *Project Gutenberg*) y produzca un informe con:

- (a) Cantidad total de caracteres, líneas y palabras del texto.

(b) La cantidad de apariciones de cada palabra distinta (normalizando a minúsculas y eliminando signos de puntuación), presentada en orden descendente de frecuencia. Para palabras con igual frecuencia, ordenar alfabéticamente.

(c) Las 20 palabras más frecuentes con su rango (posición 1 a 20) y frecuencia, para contrastar con la Ley de Zipf.

(d) Escribir el informe completo en un archivo de salida resultados.txt.

Fundamento: la Ley de Zipf

La **Ley de Zipf** (George Kingsley Zipf, 1949) establece que en un texto suficientemente largo, la frecuencia de la palabra de rango r es aproximadamente proporcional a $1/r$. Es decir, la segunda palabra más frecuente aparece aproximadamente la mitad de veces que la primera, la tercera un tercio, y así sucesivamente. Verificar si las 20 palabras más frecuentes del texto analizado se ajustan a esta predicción calculando el producto rango $\times$ frecuencia (que debería ser aproximadamente constante).

Diseño de estructuras de datos: lista de tuplas con búsqueda binaria

Dado que en la Semana 6 aún no se han introducido formalmente los diccionarios de Python (se estudian en la Semana 9), este problema se resuelve con las herramientas disponibles hasta ahora:

Paso 1 — Acumulación de frecuencias: mantener una lista de tuplas (`palabra, apariciones`) ordenada alfabéticamente por palabra. Para cada palabra del texto, usar **búsqueda binaria** para determinar si ya existe en la lista. Si existe, incrementar su contador; si no, insertarla en la posición correcta para mantener el orden (usar `lista.insert(pos, tupla)`). La búsqueda binaria garantiza $O(\log n)$ para localizar cada palabra, frente a $O(n)$ de una búsqueda lineal.

Paso 2 — Ordenar por frecuencia descendente: una vez completa la tabla de frecuencias, usar **merge sort** para reordenar la lista de tuplas por apariciones (descendente), y para tuplas con igual frecuencia, por palabra (ascendente). La **estabilidad** de merge sort es clave aquí: si primero se ordena por palabra (ya lo está del Paso 1) y luego se aplica un merge sort estable por frecuencia, las palabras con igual frecuencia quedan automáticamente en orden alfabético.

¿Por qué no usar un diccionario?

Un diccionario de Python (dict) resolvería la acumulación en $O(1)$ por palabra. Pero los diccionarios se introducen formalmente en la Semana 9. Este problema es una oportunidad pedagógica para aplicar búsqueda binaria a un problema real y para apreciar el valor de la estabilidad de merge sort. Cuando se estudien diccionarios, se podrá revisar este problema y comparar ambas soluciones — un ejemplo concreto de espiral de Bruner.

Orientaciones para la resolución

Análisis: Un texto literario típico tiene entre 5.000 y 50.000 palabras, con un vocabulario de 2.000 a 8.000 palabras distintas. La búsqueda binaria con $n = 8000$ requiere ≤ 13 comparaciones por búsqueda.

Diseño: Módulos sugeridos:

```
def normalizar(palabra):          # minúscula, sin puntuación
def buscar_palabra(tabla, palabra): # búsqueda binaria → índice o -1
def insertar_ordenado(tabla, palabra): # insert en posición correcta
def actualizar_frecuencia(tabla, palabra): # buscar + incrementar o insertar
def merge_sort_por_frecuencia(tabla): # estable, desc. por freq, asc. por
palabra
def leer_y_contar(ruta):          # lectura línea a línea
def escribir_informe(ruta, estadísticas, tabla_freq):
# escritura del informe
```

Evaluación: Para un texto en español, las palabras más frecuentes deberían ser artículos y preposiciones (de, la, el, en, y, que...). Verificar que rango $\times$ frecuencia es aproximadamente constante para las primeras 10–20 posiciones.

Problema 5: Generación y procesamiento de tickets de venta

Enunciado

Este problema tiene dos partes: (I) generar pseudoaleatoriamente un archivo de texto con registros de tickets de venta, y (II) procesarlo para obtener estadísticas comerciales.

Parte I — Generación del archivo

Escribir un programa que genere un archivo ventas.txt con la siguiente estructura:

```
TICKET 1 | 2024-03-01
P0042 | 1250.00 | 2
P0007 | 350.50 | 1
P0103 | 89.90 | 5
TICKET 2 | 2024-03-01
P0007 | 350.50 | 3
TICKET 3 | 2024-03-02
...
```

Cada ticket comienza con una línea de encabezado (número de ticket consecutivo y fecha) seguida de una cantidad variable de líneas de ítems (código de producto, precio unitario y cantidad comprada), separados por ' | '.

Parámetros de generación:

Catálogo de productos: definir una lista de 50 productos con código (P0001 a P0050) y precio unitario. Mantener la lista ordenada por código para usar búsqueda binaria al consultar precios.

Período: 30 días consecutivos (2024-03-01 a 2024-03-30).

Tickets por día: entre 8 y 25 (random.randint(8, 25)).

Ítems por ticket: entre 1 y 8 (distribución sesgada hacia pocos ítems: usar random.choices(range(1, 9), weights=[30, 25, 18, 12, 7, 4, 2, 2], k=1)[0]).

Código de producto: algunos productos se venden más que otros. Definir 10 productos «estrella» con peso 5 y 40 productos regulares con peso 1, y usar `random.choices` con esos pesos.

Cantidad comprada: entre 1 y 10 (`random.choices(range(1, 11), weights=[35, 25, 15, 10, 5, 4, 3, 1, 1, 1], k=1)[0]`).

Parte II — Procesamiento del archivo

Escribir un segundo programa que lea `ventas.txt` y calcule:

- (a) Cantidad total de tickets y de ítems vendidos.
- (b) Monto total facturado (suma de `precio × cantidad` para cada ítem de cada ticket).
- (c) Los 10 productos más vendidos (por cantidad total de unidades), ordenados de mayor a menor. Usar la misma estrategia del Problema 4: lista de tuplas (`código`, `unidades_totales`) mantenida ordenada por `código` con búsqueda binaria, luego reordenada con `merge sort` por unidades descendente.
- (d) El día con mayor facturación y el día con menor facturación.
- (e) Escribir los resultados en un archivo `reporte_ventas.txt`.

Orientaciones para la resolución

Análisis: Con los parámetros indicados, el archivo tendrá aproximadamente 500 tickets (promedio $16.5/\text{día} \times 30 \text{ días}$) y unas 1500–2000 líneas de ítems. El archivo de texto resultante ocupará alrededor de 50–80 KB.

Diseño — Parte I: Generar las fechas aritméticamente: `día_base + desplazamiento`. Para los precios, crear una lista de tuplas (`código`, `precio`) con precios realistas (usar `random.uniform(50, 5000)` redondeado a 2 decimales, generados una vez con semilla fija). El catálogo ordenado por `código` permite usar búsqueda binaria cuando se necesite consultar el precio de un producto.

Diseño — Parte II: Leer el archivo línea a línea (`for linea in archivo`). Distinguir líneas de encabezado (comienzan con 'TICKET') de líneas de ítems mediante inspección del contenido. Acumular estadísticas en listas de tuplas ordenadas, igual que en el Problema 4.

Conexión con el ciclo de vida de procesos

Cada operación de lectura/escritura sobre el archivo provoca una transición del proceso de EJECUCIÓN a BLOQUEADO. Leer línea a línea (en lugar de cargar todo con `read()`) mantiene el uso de RAM constante, lo que es crucial si el archivo de ventas fuera de varios GB en un sistema real.

Bloque D: Desafíos de extracursado

Problema 6: Estabilidad del ordenamiento — verificación experimental

Enunciado

Diseñar un experimento que verifique empíricamente la **estabilidad** (o inestabilidad) de cada algoritmo de ordenamiento implementado.

- (a) Generar una lista de 30 tuplas (nombre, nota) donde varios alumnos tengan la misma nota. Por ejemplo: [('Ana', 8), ('Bruno', 7), ('Carla', 8), ('Diego', 7), ('Elena', 9), ...]. La lista debe estar ordenada alfabéticamente por nombre.
- (b) Ordenar la lista por nota (descendente) con cada uno de los cinco algoritmos (selección, inserción, burbuja, merge sort, quick sort), sobre copias independientes.
- (c) Para cada algoritmo, verificar si los alumnos con la misma nota mantienen el orden alfabético original. Informar cuáles algoritmos son estables y cuáles no.

Orientaciones para la resolución

Diseño: Para ordenar por nota, modificar las comparaciones de cada algoritmo para comparar el segundo elemento de la tupla (`tupla[1]`). La verificación de estabilidad consiste en recorrer la lista ordenada y, para cada grupo de alumnos con igual nota, comprobar que los nombres están en orden alfabético.

Problema 7: Análisis de ventas con ranking cruzado

Enunciado

Extender el programa del Problema 5 (Parte II) para producir un análisis más detallado:

- (a) Para cada producto del catálogo, calcular: unidades totales vendidas, monto total facturado y cantidad de tickets en los que aparece.
- (b) Producir tres rankings de los 10 principales productos: por unidades vendidas, por monto facturado y por cantidad de tickets. Usar merge sort para cada ranking y comparar si los tres coinciden (¿el producto más vendido en unidades es también el más facturado?).
- (c) Calcular la facturación promedio por ticket y la desviación estándar de la facturación por ticket.

Orientaciones para la resolución

Diseño: Extender la lista de tuplas del Problema 5 a (código, unidades, monto, tickets). Mantener ordenada por código para búsqueda binaria durante la lectura. Luego, aplicar merge sort tres veces con diferentes criterios de comparación para obtener los tres rankings.

Distribución sugerida del tiempo (9 horas)

Actividad	Problemas	Tiempo estimado	Contexto
Clase práctica (3 h)	Problemas 1 y 2	~2 h	Implementación guiada con asistencia docente
	Problema 3 (inicio)	~1 h	Diseño del experimento; ejecución parcial
Extracursado (6 h)	Problema 3 (finalización)	~1 h	Completar tabla y análisis de escenarios
	Problema 4 (Zipf)	~2.5 h	Problema principal de extracursado
	Problema 5 (Tickets)	~2.5 h	Generación + procesamiento
Desafíos opcionales	Problemas 6 y 7	Variable	Para quienes completen antes o busquen profundizar

Criterio de distribución

Los Problemas 1 y 2 consolidan las implementaciones de la clase teórica y se benefician de la asistencia presencial del docente. El Problema 3, más experimental, puede iniciarse en clase y completarse fuera. Los Problemas 4 (Zipf) y 5 (Tickets) son los problemas principales de extracursado: ambos integran ordenamiento con archivos de texto y requieren diseño modular cuidadoso, justificando las 2.5 horas estimadas para cada uno. Se asignan ambos como extracursado porque (a) cada uno ejercita una faceta distinta e igualmente importante (procesamiento de texto natural vs. datos comerciales estructurados), y (b) ninguno de los dos es viable en el tiempo restante de la clase práctica.

Notas finales

Estos siete problemas cubren los contenidos prácticos de la **Semana 6** del curso. Los Problemas 1 y 2 validan la implementación correcta de los algoritmos de ordenamiento y su comparación empírica. El Problema 3 diseña un experimento de medición de tiempos que revela las diferencias entre caso promedio, mejor caso y peor caso. El Problema 4 aplica búsqueda binaria y merge sort estable a un problema de lingüística computacional. El Problema 5 integra generación pseudoaleatoria de datos, E/S de archivos, búsqueda binaria y ordenamiento en un contexto comercial realista. Los Problemas 6 y 7 profundizan en estabilidad y análisis cruzado de datos.

Criterios de evaluación formativa

Además de los criterios de semanas anteriores (metodología de Pólya, modelo de programa, *literate programming*, contratos de funciones), esta semana se evaluarán especialmente:

Correctitud de los algoritmos: cada función de ordenamiento produce resultados correctos para listas vacías, de un elemento, ya ordenadas, en orden inverso y con elementos repetidos.

Uso correcto de `for/range()` vs. `while`: bucles determinados con `for/range()`, bucles con condición compuesta o indeterminada con `while`.

Diseño experimental: semilla fija para reproducibilidad, múltiples repeticiones, tamaños de muestra suficientes.

Manejo de archivos: uso de context managers (`with`), lectura línea a línea para archivos grandes, cierre seguro de recursos.

Análisis crítico: las conclusiones se apoyan en datos medidos y se contrastan con las predicciones teóricas de la notación O grande.

Para profundizar

(1) En el Problema 4, comparar el tiempo de la solución con búsqueda binaria vs. búsqueda lineal para la acumulación de frecuencias: ¿cuánto se gana con la binaria para un texto de 50.000 palabras? (2) En el Problema 5, generar un archivo de ventas de un año completo (365 días) y verificar que el programa de procesamiento sigue funcionando correctamente con un archivo más grande. (3) Implementar la versión in-place de quick sort con la estrategia de partición de Lomuto (pivote último, dos punteros) y comparar su tiempo con la versión simplificada del Problema 3. (4) En el Problema 4, una vez completado el análisis con lista de tuplas, reimplementar usando un diccionario de Python (`dict`) y comparar la legibilidad y el rendimiento de ambas soluciones — esta comparación será formalmente motivada en la Semana 9.